

PRODUCT BACKEND ENGINEER TEST

Things to keep in mind

Results must be submitted within the time frame (48 hours) you have informed. In case of any change in test time frame, the new timeframe must be updated as soon as possible and must still be within hours of receiving the 1st email.

You may use Google, AI tools, or any online resources during the assignment. However, you must fully understand and be able to explain every technical decision in your submission. We are more interested in your engineering thinking and trade-off decisions than building many features.

You can use any technologies if you think they are necessary for your system.

Test result note

Please upload all of result into 1 folder and submit via Link (Email)

More about us:

- GEEK Up business: <https://geekup.vn>
- GEEK Up environment: <https://www.facebook.com/GEEKUpAdventure>



028 6262 4400



adventure@geekup.vn

RESPONSIBILITIES & REQUIREMENTS:

1. Business Context:

A startup is preparing to launch an online Concert Ticket Booking Platform.



The platform allows users to:

- Browse concerts
- View ticket categories and prices
- Reserve tickets
- Apply promotional vouchers
- Track booking status

To support daily operations, the startup also needs an internal Operation Dashboard for operators and administrators.

The Operation Dashboard helps the operation team:

- Monitor bookings
- Manage / publish new concerts' tickets
- Validate ticket availability
- Manage voucher campaigns
- Handle failed or suspicious bookings
- Update booking status manually when necessary

2. Business Expectation

To attract users during launch week, the startup will run a Flash Sale Campaign for a highly anticipated concert event.

Expected traffic:

- Around 50,000 users
- Peak traffic approximately 300–500 booking requests per minute

Each concert event has:

- Limited ticket quantity
- Multiple ticket categories (VIP, Standard, etc.)
- Limited promotional vouchers

The startup is concerned about:

- Overselling tickets
- Duplicate bookings caused by retries
- Users abusing promotional vouchers
- System instability during flash sale traffic spikes
-

3. Requirement

You are hired as a Software Engineer to help design and implement a simple backend system for both:

- Customer-facing booking flows
- Internal operation workflows

Your job is:

- Design an architecture for this Concert Ticket Booking Platform.
- Setup & implement the codebase that reflects the architecture
- Provide Backend APIs for both:
 - Customer-facing booking flows
 - Internal operation workflows
-

You should send back to us:

- Documents that related to System / Database Design, including your analytics & explanation about your design
- Your codebase, including:
 - Source code
 - Coding guideline & convention (how to code a new API, how to run unit test)
 - How to setup & run your codebase in local
 - API documents (ex: you can use Swagger)
 - API testing collections (ex: you can use Postman) – it should work with the local setup
- Documents that describe your **assumptions / what you have done / what you have not done** for features & scope of features that your system is serving / not serving (limitation), for example:
 - You assume that your booking ticket will have 4 states (received, waiting for payment, complete payment, ...) so you will implement an API to support change order status within these 4 states. Your
 - system supports operation team to create vouchers, but does not support update vouchers' information or delete vouchers, ...
 - You can even not serve APIs for create/update/delete vouchers from the operation dashboard, just seeding vouchers data and making sure customers can apply that voucher when creating an order.

You are **NOT** expected to build a production-ready system

Instead, we want to understand:

- How you think about backend design
- How you structure your code
- How you reason about possible issues and workflows in the system

All examples above just emphasize that you should know how to define the scope of the features that your system will serve, and that you'll have sufficient time to produce the best output. So please take your time to think carefully about what you should prioritize to build first in order to deliver the greatest value to the business.